

Software Engineering / Project Report

CODEX Neural – Multi-Agent AI Research System

A Multimodal Document Intelligence and Web Research Platform

Submitted by:

1024170298 Pranav Garg
1024170304 Harshil Agarwal
1024170160 Gurshan Singh
1024170158 Noorpreet Kaur

Group: 4 BE Third Year, CSE

Submitted to:

Dr. Reaya Grewal

Mid-Semester Evaluation

Computer Science and Engineering Department

Thapar Institute of Engineering and Technology, Patiala

Contents

1	Introduction	4
1.1	Background and Context	4
1.2	Problem Statement	4
1.2.1	Problem Scope	4
1.3	Project Objectives	5
1.4	Project Contributions	5
2	Methodology and Design	6
2.1	System Architecture	6
2.1.1	High-Level Design	6
2.1.2	Technical Stack	6
2.2	Use Case Model	7
2.2.1	Use Case Diagram	7
2.2.2	Use Case Scenarios	7
2.3	Class and Module Design	10
2.4	Activity Diagram	12
2.5	Implementation Details	14
2.5.1	Frontend Application	14
2.5.2	FastAPI API Layer	14
2.5.3	Authentication and Password Recovery	15
2.5.4	NEXUS Research Pipeline	15
2.5.5	Web Search and Scraping	15
2.5.6	Individual Document Intelligence	15
2.5.7	Retrieval-Augmented Question Answering	16
2.5.8	Administrative Dashboard	16
2.5.9	Database Design	16
3	Results and Evaluation	17
3.1	Testing Methodology	17
3.1.1	Functional Testing	17
3.1.2	Integration Testing	18
3.2	Performance Metrics	18
3.3	Analysis	18
4	Conclusion	20
4.1	Summary of Work	20
4.2	Key Findings	20
4.3	Future Work and Recommendations	20

4.4	Final Remarks	21
	Bibliography	22

Introduction

1.1 Background and Context

The rapid growth of digital information has made research increasingly dependent on the ability to locate, read, organize, and synthesize information from multiple sources. Conventional search interfaces are effective for locating individual webpages, but they generally leave the user responsible for opening sources, extracting relevant information, comparing evidence, and preparing a structured report.

CODEX Neural is an AI-assisted research and document-intelligence platform that combines two complementary workflows. The first workflow, NEXUS Research Mode, accepts a research topic and performs web-oriented information gathering followed by source processing and report synthesis. The second workflow, Individual Analysis, accepts a PDF document, extracts and segments its text, creates vector representations, and enables question answering over the uploaded document using retrieval-augmented generation (RAG).

The system is implemented as a React frontend connected to a FastAPI backend. The backend integrates authentication, SQLite, web-search and web-scraping tools, language models, Mistral embeddings, and Chroma-based vector storage. The implementation is organized around separate frontend workspaces and backend modules so that authentication, web research, document analysis, and administration can evolve independently [sup26].

1.2 Problem Statement

Researchers often face two related problems. First, web research requires repeated manual navigation across search results and webpages before useful information can be synthesized into a coherent report. Second, when a user has a long PDF such as a research paper, technical report, or study material, ordinary keyword search does not provide a conversational interface grounded in the document.

CODEX Neural addresses this problem by providing a unified research workspace in which a user can either investigate an external topic or interrogate an individual document. The system connects user-facing interaction with retrieval and language-model workflows while also providing authentication and administrative monitoring.

1.2.1 Problem Scope

The current implementation focuses on:

- authenticated access through registration and login;
- password reset using email-delivered OTP verification;
- topic-based web research using a search tool and webpage scraping;
- synthesis of gathered research into a structured report;
- PDF upload and document-aware question answering;
- chunking, embeddings, and vector retrieval through Chroma;
- an administrative dashboard for registered users and login activity.

1.3 Project Objectives

1. **Unified Research Workspace:** Provide a single interface for web-based research and individual-document analysis.
2. **Automated Research Workflow:** Reduce repetitive manual browsing by combining web search, source scraping, language-model reasoning, and report synthesis.
3. **Document-Grounded Question Answering:** Convert uploaded PDFs into searchable vector representations and answer questions using retrieved document context.
4. **Secure User Access:** Implement registration, login, password recovery, OTP verification, and frontend session persistence.
5. **Administrative Visibility:** Provide an administrator with registered-user information and recorded login activity.
6. **Modular Architecture:** Separate frontend, API, authentication, research agents, document intelligence, storage, and model responsibilities.

1.4 Project Contributions

The project combines conventional web engineering with AI-specific components. It provides a dual-mode workflow: NEXUS focuses on external information discovery and synthesis, while Individual Analysis focuses on retrieval from a user-provided document. The repository is maintained as the CODEX Neural project [sup26].

Methodology and Design

2.1 System Architecture

CODEX Neural follows a client–server architecture. The React application provides the user interface, while FastAPI exposes REST endpoints for authentication, research, document processing, and administration. The backend delegates specialized tasks to authentication functions, the research pipeline, web tools, and document-intelligence functions.

2.1.1 High-Level Design

CODEX Neural is organized into four main implementation areas: the React client, the FastAPI service layer, the AI research and document-intelligence workflows, and the persistence layer. The client communicates with the backend through REST endpoints. The backend then dispatches each request to the appropriate authentication, research, document-processing, or administration module.

The main request path is:

1. The user interacts with the React application.
2. React sends requests to the FastAPI backend.
3. FastAPI dispatches requests to authentication, research, document, or administration functionality.
4. The research workflow uses web-search and scraping tools and language-model components.
5. The document workflow extracts PDF text, creates chunks and embeddings, stores them in Chroma, and retrieves relevant chunks for question answering.
6. Authentication and administrative operations use SQLite.

2.1.2 Technical Stack

Layer	Technology / Role
Frontend	React and JavaScript; authentication screens, NEXUS workspace, Individual Analysis workspace, and admin dashboard.
Backend API	FastAPI and Pydantic request models.
Research orchestration	Python pipeline with LangChain agents.
LLM	ChatGrok using the configured GPT-OSS model for research-agent and report-writing tasks.
Search	Tavily search tool.
Web extraction	Requests and BeautifulSoup.
PDF processing	PyMuPDF (fitz).
Text splitting	RecursiveCharacterTextSplitter.
Embeddings	MistralAIEmbeddings.
Vector database	Chroma.
Relational storage	SQLite for users, OTP data, and login activity.
Email	SMTP for password-reset OTP delivery.

Table 2.1: CODEX Neural technical stack.

2.2 Use Case Model

2.2.1 Use Case Diagram

The major actors are the Research User and Administrator. External services such as Tavily Search and SMTP Email participate in specialized workflows.

2.2.2 Use Case Scenarios

UC-01: Register

- **Actor:** Research User.
- **Precondition:** No existing account with the submitted email.
- **Main Flow:** The user enters an email and password. The frontend sends the information to the registration endpoint. The backend validates the request and creates the user record in SQLite.
- **Postcondition:** A new account is available for login.

UC-02: Login

- **Actor:** Research User.
- **Precondition:** A registered account exists.

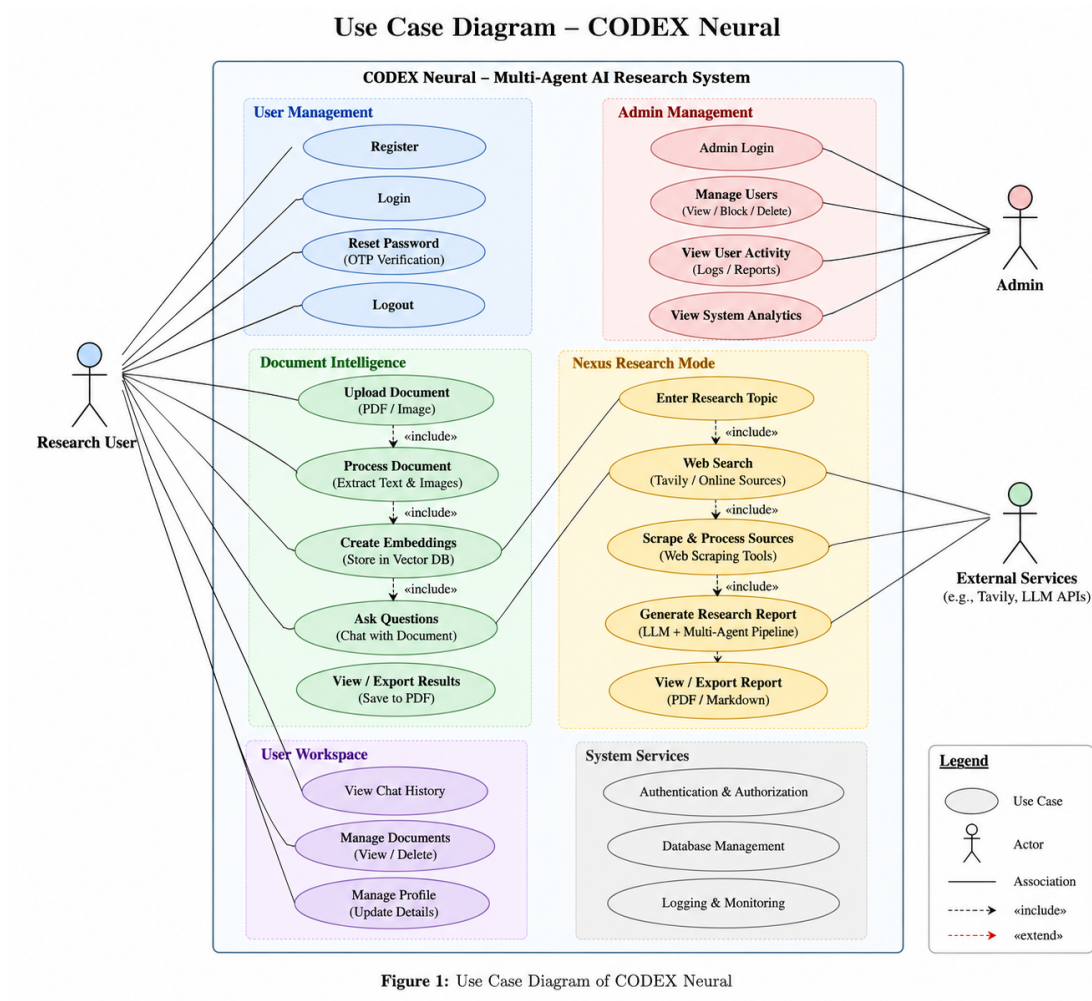


Figure 2.1: Use case diagram of CODEX Neural.

- **Main Flow:** The user submits credentials. FastAPI invokes authentication. A successful login is recorded in `login_activity`, and the frontend stores the authenticated user in session storage.
- **Postcondition:** The authenticated workspace is displayed.

UC-03: Reset Password

- **Actor:** Research User and SMTP Email Service.
- **Precondition:** The submitted email belongs to a registered account.
- **Main Flow:** The user requests a reset, receives a generated OTP by email, verifies the OTP, and submits a new password.
- **Postcondition:** The password-reset operation updates the stored password when verification succeeds.

UC-04: Research Topic

- **Actor:** Research User and Tavily Search.
- **Precondition:** The user is authenticated and provides a non-empty topic.
- **Main Flow:** The topic is sent to `/api/research`. The research agent gathers search results, a relevant URL is scraped, and the report-writing component synthesizes the gathered research.
- **Postcondition:** A structured research report is returned to the React interface.

UC-05: Analyze PDF

- **Actor:** Research User.
- **Precondition:** A readable PDF is selected.
- **Main Flow:** The PDF is uploaded through `/api/individual/upload`. Text is extracted page by page, headings are detected, documents are split into chunks, and embeddings are stored in the Chroma collection `individual_documents`.
- **Postcondition:** The document receives a document identifier and becomes available for queries.

UC-06: Ask Document Question

- **Actor:** Research User.
- **Precondition:** The selected PDF has already been indexed.
- **Main Flow:** The user submits a question through `/api/individual/chat`. Relevant chunks are retrieved using vector/MMR retrieval and filtered by document identifier. The language model generates an answer from the retrieved context.

- **Postcondition:** The answer and available page references are displayed.

UC-07: Admin Dashboard

- **Actor:** Administrator.
- **Precondition:** The authenticated account matches the configured administrator identity.
- **Main Flow:** The dashboard requests user and login-activity information from the backend.
- **Postcondition:** The administrator can view registered users, login counts, and recent authentication events.

UC-08: Manage Users and Login Activity

- **Actor:** Administrator.
- **Precondition:** The administrator dashboard is accessible.
- **Main Flow:** The administrator can remove a non-administrator user, remove an individual login record, or clear login history. Administrative endpoints verify administrator access.
- **Postcondition:** Selected database records are removed and the dashboard can be refreshed.

2.3 Class and Module Design

The Python backend is primarily organized into modules and functions rather than a large collection of explicit domain classes. Therefore, the following UML diagram represents the implemented responsibilities as conceptual software components mapped to the actual modules.

The main responsibilities are:

- **FrontendApp:** authentication state, navigation, NEXUS requests, PDF upload, and document chat.
- **AuthService:** registration, authentication, password-reset requests, OTP verification, and password reset.
- **ResearchPipeline:** topic research and report synthesis.
- **SearchAgent and WebScraper:** external information retrieval and source processing.
- **ReportWriter:** transformation of gathered research into a structured report.
- **DocumentProcessor:** PDF extraction and retrieval chunk creation.
- **VectorStore:** Chroma-based document storage and retrieval.

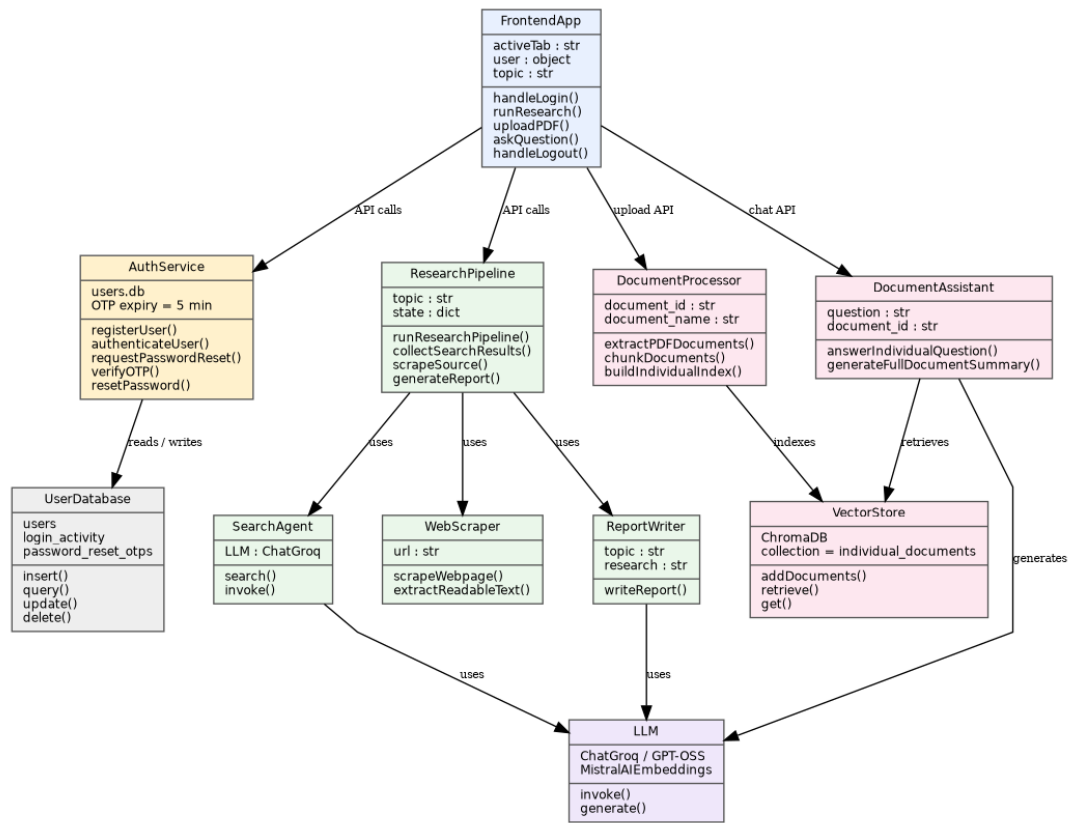


Figure 2.2: Conceptual class/module diagram mapped to the CODEX Neural implementation.

- **DocumentAssistant:** document-grounded question answering.
- **UserDatabase:** SQLite persistence for users, OTP records, and login activity.

2.4 Activity Diagram

The activity flow combines authentication with the three principal workspaces: NEXUS Research, Individual Analysis, and the administrator dashboard.

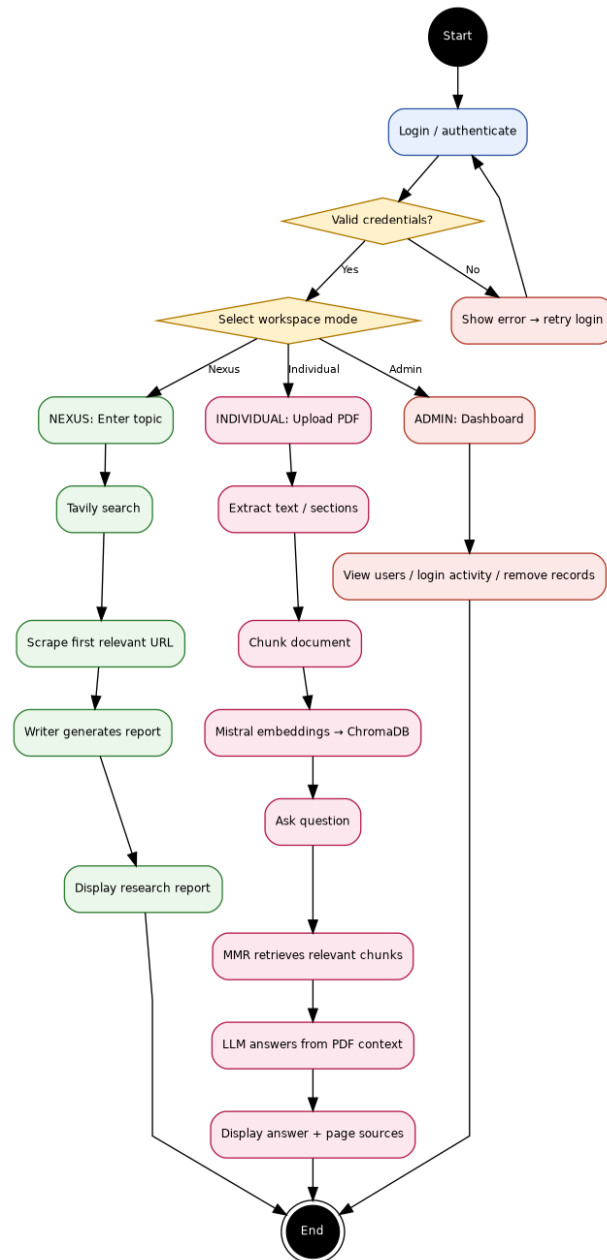


Figure 2.3: Primary activity flow of CODEX Neural.

2.5 Implementation Details

2.5.1 Frontend Application

The frontend is implemented as a React single-page application. The main application component maintains authentication state and switches between NEXUS, Individual Analysis, and the administrator dashboard. Authentication state and the active workspace are stored in browser session storage.

The login screen sends credentials to `/api/login`, while the registration screen sends account data to `/api/register`. The forgot-password workflow communicates with the OTP-related backend endpoints.

The Individual Analysis component validates PDF selection, uploads the file, displays document metadata, and provides a conversational question interface. Assistant responses are rendered as Markdown and may contain page-level source information returned by the backend.

2.5.2 FastAPI API Layer

The backend entry point is `backend/main.py`. It defines Pydantic request models and REST endpoints for authentication, research, document upload, document chat, administration, and health checking.

Endpoint	Purpose
POST <code>/api/register</code>	Create a user account.
POST <code>/api/login</code>	Authenticate a user and record login activity.
POST <code>/api/forgot-password</code>	Start password-reset OTP delivery.
POST <code>/api/verify-otp</code>	Verify the password-reset OTP.
POST <code>/api/reset-password</code>	Set a new password after verification.
POST <code>/api/research</code>	Run the NEXUS research pipeline.
POST <code>/api/individual/upload</code>	Process and index an uploaded PDF.
POST <code>/api/individual/chat</code>	Answer a question about an indexed PDF.
GET <code>/api/admin/dashboard</code>	Return administrator dashboard data.
DELETE <code>/api/admin/users/{id}</code>	Remove a non-administrator user.
DELETE <code>/api/admin/logins/{id}</code>	Remove one login record.
DELETE <code>/api/admin/logins</code>	Clear login history.
GET <code>/api/health</code>	Return backend health information.

Table 2.2: Major CODEX Neural API operations.

2.5.3 Authentication and Password Recovery

The authentication module uses SQLite persistence and hashes passwords before storage. Registration normalizes email input and rejects duplicate accounts. During login, the backend records a timestamped authentication event in `login_activity`.

Password recovery generates a six-digit OTP. The OTP is stored with an expiration time and delivered using SMTP. The user must verify the OTP before a new password is accepted.

2.5.4 NEXUS Research Pipeline

NEXUS Research Mode begins with a topic entered by the user. The backend passes the topic to the research pipeline. The research-agent component is constructed with a language model and the Tavily search tool. Search results contain source titles, URLs, and snippets. A second agent is available for webpage scraping through the `scrape_webpage` tool.

The report-writing component receives the topic and gathered research and uses a structured prompt to produce an introduction, key findings, deeper analysis, conclusion, and a limited source list.

The workflow can be represented as:

Topic → Search → Source Retrieval → Research Synthesis → Structured Report.

2.5.5 Web Search and Scraping

The search tool uses Tavily and requests up to five search results for a query. The tool extracts a title, URL, and short content snippet for each result.

The webpage-scraping tool uses HTTP requests and BeautifulSoup. Elements that generally do not contribute to article text, including scripts, styles, navigation, headers, footers, sidebars, and forms, are removed before readable page text is returned to the research workflow.

2.5.6 Individual Document Intelligence

The Individual Analysis module provides a document-specific RAG workflow. When a PDF is uploaded, the backend computes a deterministic document identifier from the file bytes and stores the PDF under the individual document storage directory.

PyMuPDF extracts page text. The implementation attempts to recognize common section-heading patterns and records metadata including document identifier, document name, page number, section, and content type.

The extracted documents are split using `RecursiveCharacterTextSplitter` with a chunk size of 3000 characters and an overlap of 200 characters. The resulting chunks

receive chunk metadata and are inserted into the Chroma vector store using Mistral embeddings.

2.5.7 Retrieval-Augmented Question Answering

When the user asks a question, the document identifier is sent together with the question. The retrieval stage uses the indexed document context and retrieves relevant chunks. The retrieved content is then provided to the language model for answer generation.

The conceptual RAG flow is:

PDF → Text Extraction → Chunking → Embeddings → Chroma

and:

Question → Vector Retrieval → Relevant Chunks → LLM → Answer.

This separates document retrieval from answer generation and allows the answer to be grounded in indexed document context.

2.5.8 Administrative Dashboard

The administrator dashboard provides operational visibility into the authentication database. The backend returns total users, total logins, registered users, and recent login records. Administrative endpoints verify the configured administrator identity before returning or modifying dashboard information. The implementation also prevents deletion of the administrator account through the user-deletion endpoint.

2.5.9 Database Design

SQLite is used for application authentication and administration data. The principal structures are:

- **users:** user identifiers, email addresses, password hashes, and registration timestamps;
- **password_reset_otp:** email, OTP, and expiration data;
- **login_activity:** user identifier, email, and login timestamp.

The document-intelligence workflow uses a separate Chroma persistence directory and collection for vectorized document chunks.

Results and Evaluation

3.1 Testing Methodology

The system is evaluated through functional and integration-oriented test cases covering the major user journeys. Because the repository does not provide a controlled benchmark dataset or reproducible performance suite, this report does not invent numerical latency or accuracy values.

3.1.1 Functional Testing

ID	Test Case	Expected Result	Status
T01	Register valid account	Account is created.	To be verified
T02	Register duplicate email	Registration is rejected.	To be verified
T03	Login valid credentials	User is authenticated and login is recorded.	To be verified
T04	Login invalid credentials	Authentication fails.	To be verified
T05	Password reset	OTP workflow completes when configured.	To be verified
T06	Upload valid PDF	PDF is extracted, chunked, embedded, and indexed.	To be verified
T07	Upload non-PDF	Request is rejected.	To be verified
T08	Ask document question	Relevant context and an answer are returned.	To be verified
T09	Research a topic	Search, source processing, and report synthesis execute.	To be verified
T10	Admin dashboard	Authorized admin receives dashboard data.	To be verified
T11	Unauthorized admin request	Backend rejects access.	To be verified
T12	Health check	Backend returns health information.	To be verified

Table 3.1: Functional test matrix.

3.1.2 Integration Testing

Integration testing focuses on boundaries between the React frontend and FastAPI backend, FastAPI and SQLite, the research pipeline and external search/scraping services, and the document pipeline and Chroma/LLM components.

Important paths include:

1. React → FastAPI authentication endpoints.
2. React → FastAPI research endpoint.
3. React → FastAPI PDF upload and chat endpoints.
4. FastAPI → SQLite authentication and administration data.
5. Research pipeline → Tavily and webpage scraping tools.
6. PDF processor → Mistral embeddings → Chroma retrieval.
7. Retrieved document context → language model → frontend answer.

3.2 Performance Metrics

No controlled benchmark results are included in the repository. The following measurements should be recorded using a fixed test environment before final submission.

Metric	Measurement Method	Recorded Value
API response time	Endpoint request duration	To be measured
PDF indexing time	Upload-to-index duration	To be measured
Question-answer time	Question-to-answer duration	To be measured
Research generation time	Topic-to-report duration	To be measured
Memory usage	Backend process measurement	To be measured
Document scalability	Increasing PDF sizes	To be measured

Table 3.2: Performance evaluation plan.

3.3 Analysis

The implementation demonstrates a complete application path from a React user interface through a REST backend to AI-oriented research and document retrieval services. The separation of NEXUS and Individual Analysis allows two different research tasks to use different retrieval strategies.

The Individual Analysis workflow is structured as extraction, section-aware meta-data generation, recursive chunking, embeddings, vector retrieval, and language-model generation. The NEXUS workflow separates search, source processing, and report synthesis.

The current implementation also has deployment considerations. Authentication is application-level, administrator access uses a configured administrator identity, the API configuration permits broad CORS access, and several workflows depend on external AI, search, email, and hosting services. These points should be considered during production hardening rather than presented as already solved production-security or scalability guarantees.

Conclusion

4.1 Summary of Work

CODEX Neural implements a multi-component AI research platform combining web research and document intelligence in one React and FastAPI application. The system provides user registration and authentication, password recovery through OTP verification, a NEXUS research workflow, individual PDF question answering, and an administrator dashboard.

The NEXUS workflow combines web search, source scraping, language-model agents, and structured report writing. The Individual Analysis workflow combines PDF extraction, section-aware metadata, recursive chunking, Mistral embeddings, Chroma retrieval, and language-model generation.

4.2 Key Findings

- **Dual research modes:** external topic research and document-specific analysis are available in one application.
- **Retrieval-based document intelligence:** uploaded PDFs are transformed into searchable chunks and vector representations before question answering.
- **Agent-oriented research:** search, webpage processing, and report writing are separated into specialized components.
- **Integrated authentication:** registration, login, OTP password recovery, and login-activity tracking are connected to the application.
- **Administrative visibility:** the dashboard exposes registered-user information and authentication activity through protected endpoints.

4.3 Future Work and Recommendations

1. **Scalable document storage:** move from local Chroma persistence and SQLite toward managed storage for multiple concurrent users and large document collections.
2. **Document isolation:** introduce explicit user-level collections or namespaces for multi-user deployments.
3. **Authentication hardening:** introduce role-based authorization, stronger session management, and production-grade credential handling.

4. **Observability:** add structured logs, request tracing, latency monitoring, and failure metrics.
5. **Research evaluation:** create benchmark questions/documents and measure retrieval quality, grounding, latency, and report quality.
6. **Large-document processing:** introduce asynchronous ingestion, caching, background processing, and object storage.
7. **Additional modalities:** extend support to tables, images, scanned PDFs, OCR, and other document formats.
8. **Deployment hardening:** restrict CORS origins, protect secrets, use HTTPS, and separate development and production configuration.
9. **Research history and export:** extend persistent report history, PDF export, and collaboration features.

4.4 Final Remarks

CODEx Neural demonstrates how a conventional web application can be extended with retrieval, agent tooling, embeddings, and language-model generation to support research-oriented tasks. The current architecture establishes separate paths for external web research and document-grounded analysis while maintaining a shared authentication and administration layer.

Bibliography

- [Chr26] Chroma. *Chroma Documentation*. Chroma. 2026. URL: <https://docs.trychroma.com/>.
- [Fas26] FastAPI. *FastAPI Documentation*. FastAPI. 2026. URL: <https://fastapi.tiangolo.com/>.
- [Lan26] LangChain. *LangChain Documentation*. LangChain. 2026. URL: <https://python.langchain.com/>.
- [LPP+20] Patrick Lewis, Ethan Perez, Aleksandra Piktus, et al. “Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks”. In: *Advances in Neural Information Processing Systems* 33 (2020).
- [Mis26] Mistral AI. *Mistral AI Documentation*. Mistral AI. 2026. URL: <https://docs.mistral.ai/>.
- [PyM26] PyMuPDF Developers. *PyMuPDF Documentation*. PyMuPDF. 2026. URL: <https://pymupdf.readthedocs.io/>.
- [Rea26] React Team. *React Documentation*. Meta Open Source. 2026. URL: <https://react.dev/>.
- [Ric26] Leonard Richardson. *Beautiful Soup Documentation*. Beautiful Soup. 2026. URL: <https://www.crummy.com/software/BeautifulSoup/bs4/doc/>.
- [SQL26] SQLite Consortium. *SQLite Documentation*. SQLite. 2026. URL: <https://www.sqlite.org/docs.html>.
- [sup26] supremeinferno. *CODEX Neural – Multi-Agent AI Research System*. GitHub. 2026. URL: <https://github.com/supremeinferno/codex-neural> (cit. on pp. 4, 5).
- [Tav26] Tavily. *Tavily Documentation*. Tavily. 2026. URL: <https://docs.tavily.com/>.